ユーザー追加ダイアログUI設計書

メタデータ

title: ユーザー追加ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-24
author: Development Team
tags: [system-admin, user-management, dialog, add-form]

関連ドキュメント

- ユーザー管理画面
- ユーザー編集ダイアログ
- ユーザー管理API仕様
- UIポリシー

---

1. 概要

1.1. 目的

システム管理者が新規ユーザーを追加するためのモーダルダイアログです。ユーザーID（メールアドレス）、ユーザー名、パスワード、所属部署、ロール、二段階認証設定、アクティブ状態を入力して登録します。

1.2. 表示条件

- ユーザー管理画面の「追加」ボタンをクリック

- System Admin権限が必要

1.3. 画面イメージ

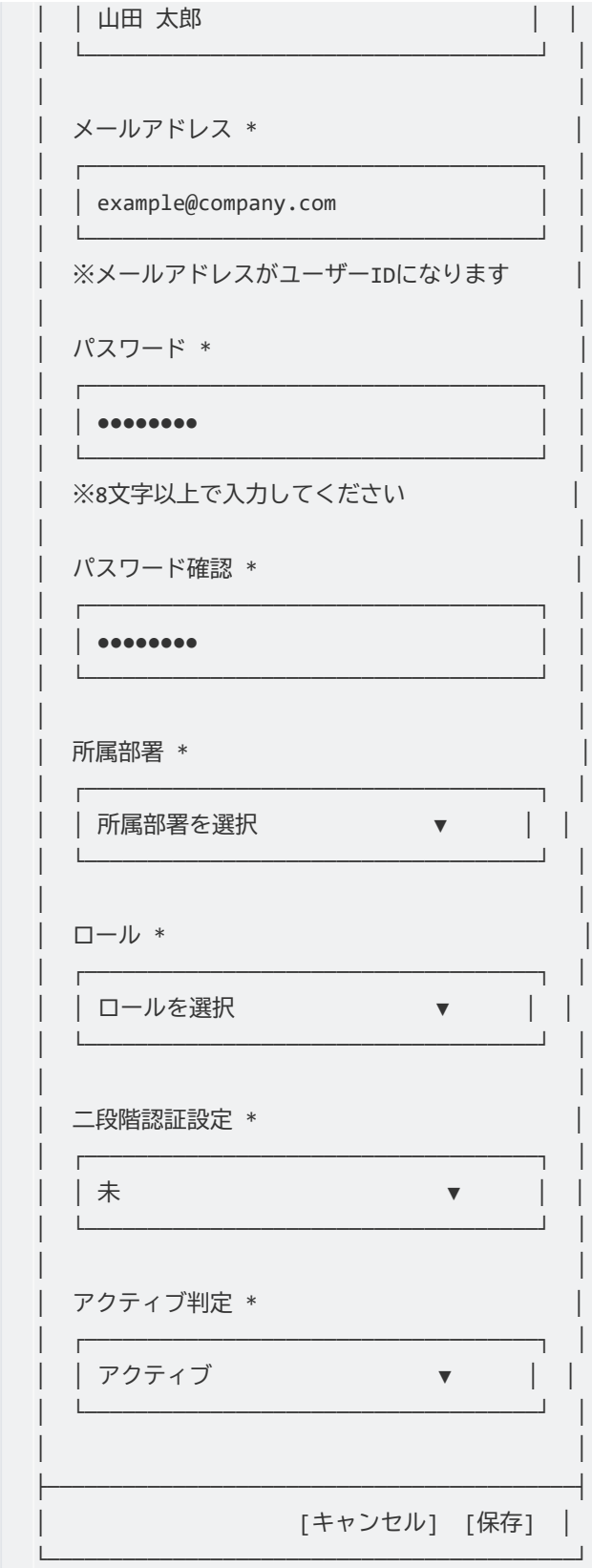


---

2. ダイアログ構成

2.1. レイアウト

```
┌──────────────────────────────────────┐
│ ユーザー追加                    [×]  │
├──────────────────────────────────────┤
│                                      │
│  ユーザー名 *                        │
│  ┌──────────────────────────────┐    │
```



2.2. モーダル仕様

コンポーネント: `<Modal>`

仕様:

項目	値
タイトル	「ユーザー追加」
幅	600px
位置	画面中央
マスク	背景を暗くする
キーボードESC	閉じる（確認なし）
マスククリック	閉じない

---

3. 入力項目

3.1. ユーザー名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ユーザー名」
必須	○
プレースホルダー	「山田 太郎」
最大文字数	30文字
初期値	空文字列

バリデーション:

```
{
  required: true,
  message: 'ユーザー名を入力してください'
}
```

3.2. メールアドレス（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「メールアドレス」
必須	○
プレースホルダー	「example@company.com」
型	email
初期値	空文字列
重複チェック	あり（API呼び出し時）

注意事項:

- メールアドレスがuser_idとして使用される
- organization_userテーブルとuser_authenticationテーブルの両方に登録される

バリデーション:

```
{
  required: true,
  message: 'メールアドレスを入力してください'
},
{
  type: 'email',
  message: '正しいメールアドレスを入力してください'
}
```

重複チェック:

- API呼び出し時（createUser）に重複をチェック
- 重複の場合: HTTPステータス409
- エラーメッセージ:「入力されたユーザーID（メールアドレス）は既に存在しています」

3.3. パスワード（必須）

コンポーネント: `<Form.Item>` + `<Input.Password>`

仕様:

項目	値
ラベル	「パスワード」
必須	○
プレースホルダー	「8文字以上のパスワード」
最小文字数	8文字
可視化トグル	有効
初期値	なし

注意事項:

- **追加時のみ**パスワード入力欄が表示される
- 編集時はパスワード変更不可
- バックエンドでbcryptハッシュ化して保存

バリデーション:

```
{
  required: true,
  message: 'パスワードを入力してください'
},
{
  min: 8,
  message: 'パスワードは8文字以上で入力してください'
}
```

3.4. パスワード確認（必須）

コンポーネント: `<Form.Item>` + `<Input.Password>`

仕様:

項目	値
ラベル	「パスワード確認」

必須	○
プレースホルダー	「パスワードを再入力」
可視化トグル	有効
初期値	なし
依存フィールド	password

バリデーション:

```
{
  required: true,
  message: 'パスワード確認を入力してください'
},
{
  validator: ({ getFieldValue }) => ({
    validator(_, value) {
      if (!value || getFieldValue('password') === value) {
        return Promise.resolve();
      }
      return Promise.reject(new Error('パスワードが一致しません'));
    },
  }),
}
```

3.5. 所属部署（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「所属部署」
必須	○
プレースホルダー	「所属部署を選択」
データソース	departmentApiService.fetchMasters()
初期値	なし

選択肢:

```
[
  { value: 'DEPT001', label: '開発部' },
  { value: 'DEPT002', label: '営業部' },
  // ...
]
```

バリデーション:

```
{
  required: true,
  message: '所属部署を選択してください'
}
```

3.6. ロール（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「ロール」
必須	○
プレースホルダー	「ロールを選択」
データソース	roleApiService.fetchMasters()
初期値	なし

選択肢:

```
[
  { value: 'ROLE001', label: 'プロバイダー' },
  { value: 'ROLE002', label: 'システム管理者' },
  { value: 'ROLE003', label: 'データ管理者' },
  { value: 'ROLE004', label: '実験管理者' },
  { value: 'ROLE005', label: '実験作業者' },
  { value: 'ROLE006', label: 'MI技術者' },
]
```

バリデーション:

```
{
  required: true,
  message: 'ロールを選択してください'
}
```

3.7. 二段階認証設定（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「二段階認証設定」
必須	○
初期値	0（未）

選択肢:

```
[
  { value: 0, label: '未' },
  { value: 1, label: '済' },
]
```

注意事項:

- user_authenticationテーブルのentry_flgに対応
- 0 = false（未設定）、1 = true（設定済み）

バリデーション:

```
{
  required: true,
  message: '二段階認証設定を選択してください'
}
```

3.8. アクティブ判定（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「アクティブ判定」
必須	○
初期値	true（アクティブ）

選択肢:

```
[
  { value: true, label: 'アクティブ' },
  { value: false, label: '非アクティブ' },
]
```

バリデーション:

```
{
  required: true,
  message: 'アクティブ判定を選択してください'
}
```

---

4. アクションボタン

4.1. 配置

位置: モーダルフッター（右寄せ）

レイアウト:

```
<Space>
  <Button onClick={handleModalCancel}>キャンセル</Button>
```

```
    <Button type="primary" onClick={handleModalOk} loading={loading}>保存</Button>
  </Space>
```

4.2. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`

動作:

1. モーダルを閉じる
2. フォームをリセット
3. 確認なし

4.3. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
ローディング	送信中は `true`
無効化条件	loading=true

動作フロー:

1. フォームバリデーション実行
2. バリデーションエラーがある場合、エラーメッセージ表示
3. ローディング状態に変更
4. **トランザクション1**: organization_userテーブルにINSERT

5. **トランザクション2**: user_authenticationテーブルにINSERT（パスワードハッシュ化）
6. 成功時:
    - モーダルを閉じる
    - データ再読み込み
    - 成功メッセージ表示: `"ユーザーを追加しました"`
7. 失敗時:
    - エラーメッセージ表示
    - モーダルは開いたまま

実装例:

```
const handleModalOk = async () => {
  try {
    const values = await form.validateFields();

    const createData: CreateUserRequest = {
      userId: values.mailAddress,           // メールアドレス = ユーザーID
      userName: values.userName,
      mailAddress: values.mailAddress,
      departmentCd: values.departmentCd,
      roleCd: values.roleCd,
      twoFactorAuth: values.twoFactorAuth === 1,  // number → boolean
      isActive: values.isActive,
      password: values.password,            // フォームから入力されたパスワード
    };

    setLoading(true);
    await createUser(createData);
    await loadUsers();
    message.success('ユーザーを追加しました');

    setIsModalVisible(false);
    form.resetFields();
  } catch (error) {
    console.error('保存に失敗:', error);
    const errorMessage = error instanceof Error ? error.message : '保存に失敗しました';
    message.error(errorMessage);
  } finally {
    setLoading(false);
```

```
  }
};
```

---

5. API仕様

5.1. ユーザー新規作成

エンドポイント: `POST /api/v1/systemManagement/user`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "userId": "newuser@example.com",
  "userName": "新規ユーザー",
  "mailAddress": "newuser@example.com",
  "departmentCd": "DEPT001",
  "roleCd": "ROLE003",
  "twoFactorAuth": false,
  "isActive": true,
  "password": "TempPassword123!"
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "userId": "newuser@example.com",
    "userName": "新規ユーザー",
    "mailAddress": "newuser@example.com",
    "departmentCd": "DEPT001",
    "departmentName": "開発部",
```

```
    "roleCd": "ROLE003",
    "roleName": "実験管理者",
    "twoFactorAuth": false,
    "isActive": true,
    "createAt": "2026-02-24T08:48:00Z",
    "updateAt": "2026-02-24T08:48:00Z"
  },
  "message": "ユーザーを作成しました"
}
```


レスポンス（エラー時 - 重複）:


```
{
  "success": false,
  "error": {
    "code": "RESOURCE_ALREADY_EXISTS",
    "message": "入力されたユーザーID（メールアドレス）は既に存在しています"
  },
  "meta": {
    "timestamp": "2026-02-24T08:48:00Z",
    "requestId": "req_abc123"
  }
}
```


---

6. バリデーション

6.1. クライアントサイドバリデーション

必須チェック:

- ユーザー名
- メールアドレス
- パスワード
- パスワード確認
- 所属部署
- ロール
- 二段階認証設定

- アクティブ判定

形式チェック:

- メールアドレス: email形式
- パスワード: 最小8文字
- パスワード確認: passwordフィールドと一致

6.2. サーバーサイドバリデーション

重複チェック:

- ユーザーID（メールアドレス）が既存のユーザーと重複していないか

データ整合性チェック:

- 所属部署CDが有効か
- ロールCDが有効か

---

7. エラーハンドリング

7.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`VALIDATION_ERROR`	「ユーザー名を入力してください」	フォームフィールド
`VALIDATION_ERROR`	「メールアドレスを入力してください」	フォームフィールド
`VALIDATION_ERROR`	「正しいメールアドレスを入力してください」	フォームフィールド
`VALIDATION_ERROR`	「パスワードを入力してください」	フォームフィールド
`VALIDATION_ERROR`	「パスワードは8文字以上で入力してください」	フォームフィールド
`VALIDATION_ERROR`	「パスワードが一致しません」	フォームフィールド

`RESOURCE_ALREADY_EXISTS`	「入力されたユーザーID（メールアドレス）は既に存在しています」	トースト
`SYSTEM_ERROR`	「保存に失敗しました」	トースト

7.2. エラー表示方法

フォームフィールドエラー:

```
form.setFields([
  {
    name: 'mailAddress',
    errors: ['入力されたユーザーID（メールアドレス）は既に存在しています'],
  },
]);
```

トーストメッセージ:

```
message.error('入力されたユーザーID（メールアドレス）は既に存在しています');
```

---

8. ユーザーフロー

8.1. 正常フロー

```
[ユーザー管理画面]
    ↓ 「追加」ボタンクリック
[ユーザー追加ダイアログ] 表示
    フォームリセット
    初期値設定: isActive=true, twoFactorAuth=0
    ↓ ユーザー名入力: "山田 太郎"
    ↓ メールアドレス入力: "yamada@example.com"
    ↓ パスワード入力: "SecurePassword123!"
    ↓ パスワード確認入力: "SecurePassword123!"
    ↓ 所属部署選択: "開発部"
    ↓ ロール選択: "実験管理者"
    ↓ 二段階認証設定: "未"（デフォルト）
    ↓ アクティブ判定: "アクティブ"（デフォルト）
```

```
    ↓ 「保存」ボタンクリック
フォームバリデーション実行
    ↓ OK
API呼び出し: createUser()
    トランザクション1: organization_userテーブルにINSERT
    トランザクション2: user_authenticationテーブルにINSERT（パスワードハッシュ化）
    ↓ 成功
[ユーザー管理画面]
    データ再読み込み
    成功メッセージ表示: "ユーザーを追加しました"
```

8.2. エラーフロー（重複）

```
[ユーザー追加ダイアログ]
    ↓ メールアドレス入力: "user001@example.com"（既存のメールアドレス）
    ↓ その他の項目を入力
    ↓ 「保存」ボタンクリック
フォームバリデーション実行
    ↓ OK
API呼び出し: createUser()
    ↓ エラー（409 Conflict）
[ユーザー追加ダイアログ]
    エラーメッセージ表示: "入力されたユーザーID（メールアドレス）は既に存在しています"
    モーダルは開いたまま
    ↓ メールアドレスを修正
    ↓ 再度「保存」ボタンクリック
```

8.3. エラーフロー（パスワード不一致）

```
[ユーザー追加ダイアログ]
    ↓ パスワード入力: "Password123!"
    ↓ パスワード確認入力: "Password456!"（異なる）
    ↓ 「保存」ボタンクリック
フォームバリデーション実行
    ↓ エラー
[ユーザー追加ダイアログ]
    パスワード確認フィールドにエラー表示: "パスワードが一致しません"
    ↓ パスワード確認を修正
    ↓ 再度「保存」ボタンクリック
```

---

9. データベース登録処理

9.1. 2テーブル同時登録

トランザクション1: organization_userテーブル

```
INSERT INTO organization_user (
  organization_id,
  user_id,
  user_name,
  mail_address,
  department_cd,
  role_cd,
  is_active,
  create_user_id,
  update_user_id
) VALUES (
  $1, -- organization_id (authMiddlewareから取得)
  $2, -- user_id (メールアドレス)
  $3, -- user_name
  $4, -- mail_address
  $5, -- department_cd
  $6, -- role_cd
  $7, -- is_active
  $8, -- create_user_id (ログインユーザーID)
  $9  -- update_user_id (ログインユーザーID)
);
```

トランザクション2: user_authenticationテーブル

```
INSERT INTO user_authentication (
  organization_id,
  user_id,
  password,
  entry_flg
) VALUES (
  $1, -- organization_id
  $2, -- user_id (メールアドレス)
  $3, -- password (bcryptハッシュ化済み)
  $4  -- entry_flg (twoFactorAuth)
);
```

9.2. パスワードハッシュ化

使用ライブラリ: bcrypt

ソルトラウンド: 12（デフォルト）

実装例:

```
import bcrypt from 'bcrypt';

const hashedPassword = await bcrypt.hash(plainPassword, 12);
```

---

10. アクセシビリティ

10.1. キーボード操作

Tab順序:

1. ユーザー名入力
2. メールアドレス入力
3. パスワード入力
4. パスワード確認入力
5. 所属部署選択
6. ロール選択
7. 二段階認証設定選択
8. アクティブ判定選択
9. キャンセルボタン
10. 保存ボタン

ショートカットキー:

- Enter: 保存ボタンのクリック（フォーカスが入力フィールドにない場合）
- Escape: モーダルを閉じる

10.2. ARIA属性

```
<Modal
  title="ユーザー追加"
  open={isModalVisible}
  onCancel={handleModalCancel}
  aria-labelledby="user-add-dialog-title"
  aria-describedby="user-add-dialog-description"
>
  <Form form={form} layout="vertical">
    <Form.Item
      label="ユーザー名"
      name="userName"
      rules={[{ required: true }]}
    >
      <Input
        aria-label="ユーザー名"
        aria-required="true"
      />
    </Form.Item>
  </Form>
</Modal>
```

---

11. セキュリティ考慮事項

11.1. パスワード管理

- パスワードはbcryptでハッシュ化して保存
- ソルトラウンド: 12
- APIレスポンスにパスワードを含めない
- 最小8文字を強制

11.2. マルチテナント分離

- organization_idは自動付与（authMiddleware）
- 他組織のユーザーは作成不可

11.3. 監査ログ

- create_user_idにログインユーザーIDを記録
- 作成日時を自動記録

---

12. 編集ダイアログとの違い

項目	追加ダイアログ	編集ダイアログ
タイトル	「ユーザー追加」	「ユーザー編集」
メールアドレス	入力可能	読み取り専用
パスワード入力欄	あり（必須）	なし
パスワード確認入力欄	あり（必須）	なし
初期値	空	選択したユーザーの情報
APIエンドポイント	`POST /api/v1/systemManagement/user`	`PUT /api/v1/systemManagement/user/:userId`
DB操作	2テーブルINSERT	1テーブルUPDATE

---

13. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-24	初版作成	Development Team